

HTML Media Tags

A Student Guide to Images, Audio, Video & Embedded Content — Revised Edition

Class Handout • Web Development Basics

How to read this guide: black text is everything you've already covered, unchanged. **Teal text** marks what's new in this revision — read those parts first if you're short on time.

What's Inside

- Introduction — what media tags are and why they matter
- NEW — the rule behind closing tags
- The `<img>` tag + NEW attributes
- NEW — the `<picture>` tag
- The `<audio>` tag + NEW: why `<source>` exists, NEW attributes
- The `<video>` tag + NEW attributes
- The `<source>` tag
- The `<track>` tag
- The `<embed>` tag
- The `<object>` tag + NEW attributes
- The `<iframe>` tag + NEW attributes
- NEW — MIME types in plain terms
- NEW — choosing between embed, object, and iframe
- Quick comparison table
- Common mistakes to avoid
- Practical 1 — build a multimedia profile page
- NEW — Practical 2 — debugging lab
- NEW — Practical 3 — level-up build

Introduction: What Are Media Tags?

Media tags are special HTML elements that let you add pictures, sound, and video straight into a webpage. Before HTML5, browsers needed extra plugins — like Adobe Flash — just to play audio or video. Today, media tags handle all of this natively, meaning every modern browser understands them without installing anything extra.

In this guide, you will learn what each media tag does, every attribute it can take, and how to combine all of them into one working webpage.

NEW — The One Rule Behind Closing Tags

Instead of memorising "img: no closing tag, audio: yes, source: no..." as separate facts, use one test: a tag needs a closing tag only if it can hold something inside it — child tags, or fallback content for when the browser can't display it. If a tag is always empty, it never gets one.

Attribute	Type / Values	What It Does
<code></code>	No	Empty tag — nothing goes inside it.
<code><picture></code>	Yes	Wraps <code><source></code> tags and a fallback <code></code> .
<code><audio></code>	Yes	Holds <code><source></code> tags and fallback text.
<code><video></code>	Yes	Holds <code><source></code> , <code><track></code> , and fallback text.
<code><source></code>	No	Empty tag — never used on its own.
<code><track></code>	No	Empty tag — always sits inside <code><video></code> .
<code><embed></code>	No	Empty tag — can't hold fallback content.
<code><object></code>	Yes	Holds fallback content for unsupported browsers.

Attribute	Type / Values	What It Does
<iframe>	Yes	Can hold fallback text for old browsers.

1. — The Image Tag

Definition

The tag embeds a picture into a webpage. It is a self-closing tag, which means you never write a separate closing tag like — everything is written inside one single tag.

Syntax

```

```

Attributes

Attribute	Type / Values	What It Does
src	URL / file path	Location of the image file. Required.
alt	Text	Backup text shown if the image fails to load, and read aloud by screen readers. Required for accessibility.
width	Number (pixels)	Sets the display width of the image.
height	Number (pixels)	Sets the display height of the image.
loading	"lazy" or "eager"	"lazy" delays loading the image until it is about to appear on screen, which speeds up the page.
title	Text	Extra tooltip text that appears when the mouse hovers over the image.

Example

```

```

Tip: Always fill in the alt attribute. It is one of the most important accessibility habits in HTML.

NEW — More Attributes You Should Know

These attributes solve a real problem: the same image looking wrong — too small, too large, or oddly cropped — on different screen sizes.

Attribute	Type / Values	What It Does
srcset	List of files + width/density	Offers several versions of the same image (different sizes) so the browser can pick the best fit for the screen.
sizes	CSS-style size list	Tells the browser how wide the image will actually display at different screen widths, so it can choose correctly from srcset.
crossorigin	"anonymous" / "use-credentials"	Needed when the image comes from another domain and your page needs to read its pixel data (e.g. on a <canvas>).
referrerpolicy	e.g. "no-referrer"	Controls how much referrer information is sent when the browser fetches the image.
decoding	"sync" / "async" / "auto"	Hints whether the browser can decode the image off the main thread, which can make the page feel smoother.

NEW — Example

```

```

Tip: srcset lists the files you have; sizes tells the browser how big the image will actually be shown. The browser combines both to pick the smallest file that still looks sharp.

NEW — 1B. <picture> — The Responsive Image Tag

NEW — Definition

The <picture> tag lets you show a genuinely different image — not just a different size — depending on the screen or format support. Common uses: a cropped portrait on mobile vs. a wide landscape on desktop, or a modern WebP file with a JPG fallback for older browsers. It always needs a closing tag because it wraps <source> tags and a required .

NEW — Syntax

```
<picture>
  <source media="(max-width: 600px)" srcset="portrait.jpg">
  <source srcset="landscape.jpg">
  
</picture>
```

NEW — Attributes (on the <source> tags inside)

Attribute	Type / Values	What It Does
media	Media query	Tells the browser to use this source only under certain conditions, like screen width.
srcset	File path(s)	The image file(s) to use if this source is selected.
type	MIME type	The image format, e.g. "image/webp", so the browser can skip formats it can't display.

NEW — Example

```
<picture>
  <source srcset="photo.webp" type="image/webp">
  <source srcset="photo.jpg" type="image/jpeg">
  
</picture>
```

Tip: The final inside <picture> is not optional — it's the fallback every browser can fall back to, even ones that don't understand <picture> at all.

2. <audio> — The Audio Tag

Definition

The <audio> tag embeds sound content, such as music, a podcast, or a voice recording. Unlike , it needs both an opening and a closing tag, because other tags — like <source> — are written inside it.

Syntax

```
<audio controls>
  <source src="theme-song.mp3" type="audio/mpeg">
  <source src="theme-song.ogg" type="audio/ogg">
</audio>
```

Attributes

Attribute	Type / Values	What It Does
src	URL / file path	Path to the audio file. Can be used directly instead of a <source> tag.

Attribute	Type / Values	What It Does
controls	Boolean	Shows the play, pause, and volume buttons.
autoplay	Boolean	Starts playing automatically when the page loads. Most browsers block this unless the audio is also muted.
loop	Boolean	Replays the audio automatically once it finishes.
muted	Boolean	Starts the audio muted.
preload	"auto" / "metadata" / "none"	Tells the browser how much of the file to load in advance.

Example

```
<audio controls>
  <source src="theme-song.mp3" type="audio/mpeg">
  <source src="theme-song.ogg" type="audio/ogg">
  Your browser does not support the audio element.
</audio>
```

Tip: Boolean attributes don't need a value — just writing the word turns the feature on, e.g. `<audio controls loop>`.

NEW — Why `<source>` Exists

Different browsers support different audio and video formats — one browser might play MP3 but not OGG, another the reverse. `<source>` lets you list several formats for the same clip; the browser checks each one in order and plays the first format it understands. Without it, if your one file's format isn't supported, the visitor sees nothing at all — that's why the syntax example above lists two formats, not one.

NEW — More `<audio>` Attributes: `crossorigin`

Attribute	Type / Values	What It Does
crossorigin	"anonymous" / "use-credentials"	Needed when the audio file comes from another domain and your page's JavaScript needs to access it directly (e.g. for a visualiser).

3. `<video>` — The Video Tag

Definition

The `<video>` tag works just like `<audio>`, but for video content. It plays clips directly in the browser and supports the same idea of listing several file formats using nested `<source>` tags.

Syntax

```
<video controls poster="preview.jpg">
  <source src="filename.mp4" type="video/mp4">
</video>
```

Attributes

Attribute	Type / Values	What It Does
src	URL / file path	Path to the video file. Can be used directly instead of a <code><source></code> tag.
controls	Boolean	Shows the play, pause, volume, and fullscreen controls.
autoplay	Boolean	Starts playing automatically. Browsers usually require muted to also be set.
loop	Boolean	Replays the video automatically once it finishes.
muted	Boolean	Starts the video muted.
poster	URL / file path	An image shown before the video is played.
width / height	Number (pixels)	Sets the display size of the video player.

Attribute	Type / Values	What It Does
preload	"auto" / "metadata" / "none"	Tells the browser how much of the file to load in advance.

Example

```
<video width="640" height="360" controls poster="thumbnail.jpg">
  <source src="lesson.mp4" type="video/mp4">
  <source src="lesson.webm" type="video/webm">
  Your browser does not support the video tag.
</video>
```

Tip: Use the poster attribute to show a clean preview image instead of a blank black box before playback starts.

NEW — More <video> Attributes You Should Know

The first one below is the attribute most students discover the hard way — by testing their page on a phone.

Attribute	Type / Values	What It Does
playsinline	Boolean	On mobile Safari, lets the video play inline in the page instead of forcing fullscreen the moment it starts.
crossorigin	"anonymous" / "use-credentials"	Needed when the video comes from another domain and the page needs to access its data directly (e.g. drawing frames to a <canvas>).
disablepictureinpicture	Boolean	Prevents the browser from offering its built-in picture-in-picture button.
disableremoteplayback	Boolean	Hides casting controls (e.g. Chromecast) for this video.

Tip: Try <video autoplay controls> without muted and without playsinline, then test it — on desktop the sound gets blocked, on an iPhone the video jumps to fullscreen. Both are the browser protecting the user, not a bug in your code.

4. <source> — The Source Tag

Definition

The <source> tag is written inside <audio> or <video> (or <picture>) to offer more than one file format of the same media. The browser checks each <source> in order and plays the first format it understands. It is a self-closing tag and is never used by itself.

Syntax

```
<source src="filename.mp4" type="video/mp4">
```

Attributes

Attribute	Type / Values	What It Does
src	URL / file path	Location of that particular media file. Required.
type	MIME type	Tells the browser what kind of file it is, e.g. "video/mp4" or "audio/mpeg", so it can skip formats it can't play.
media	Media query	Optional. Used to tell the browser to use this source only under certain conditions, such as screen size.

Example

```
<video controls>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.webm" type="video/webm">
</video>
```

Tip: Different browsers support different video and audio formats, so listing 2-3 formats keeps your media working everywhere. See "Why <source> Exists" above for the full reasoning.

5. <track> — The Track Tag

Definition

The <track> tag adds a text track — such as subtitles or captions — to a <video>. It is a self-closing tag placed inside <video>, and it is one of the most important tags for accessibility.

Syntax

```
<track src="captions.vtt" kind="subtitles"
      srclang="en" label="English">
```

Attributes

Attribute	Type / Values	What It Does
src	URL / file path	Path to the subtitle/caption file (usually a .vtt file). Required.
kind	subtitles / captions / descriptions / chapters / metadata	The type of text track being added.
srclang	Language code, e.g. "en"	The language the track is written in.
label	Text	The name shown to the user when choosing a track, e.g. "English".
default	Boolean	Marks this track as the one turned on automatically.

Example

```
<video controls>
  <source src="lesson.mp4" type="video/mp4">
  <track src="captions_en.vtt" kind="captions"
        srclang="en" label="English" default>
</video>
```

6. <embed> — The Embed Tag

Definition

The <embed> tag is a general-purpose way to embed an external file or resource — such as a PDF — directly into the page. It is a self-closing tag and has no way to show fallback content.

Syntax

```
<embed src="file.pdf" type="application/pdf" width="600"
      height="400">
```

Attributes

Attribute	Type / Values	What It Does
src	URL / file path	Location of the file being embedded. Required.
type	MIME type	The kind of file being embedded, e.g. "application/pdf".
width / height	Number (pixels)	Sets the display size of the embedded content.

Example

```
<embed src="resume.pdf" type="application/pdf">
```

```
width="600" height="500">
```

7. <object> — The Object Tag

Definition

The <object> tag is similar to <embed>, but more flexible — it can show fallback content, like a message or a download link, if the browser cannot display the embedded file. Because it can hold fallback content inside it, it needs a closing tag.

Syntax

```
<object data="file.pdf" type="application/pdf"
  width="600" height="400">
  fallback content here
</object>
```

Attributes

Attribute	Type / Values	What It Does
data	URL / file path	Location of the file being embedded. Required.
type	MIME type	The kind of file being embedded, e.g. "application/pdf".
width / height	Number (pixels)	Sets the display size of the embedded content.

Example

```
<object data="resume.pdf" type="application/pdf"
  width="600" height="500">
  <p>Your browser can't show this PDF.
  <a href="resume.pdf">Download it here</a>.</p>
</object>
```

NEW — More <object> Attributes: name and <param>

Attribute	Type / Values	What It Does
name	Text	Gives the embedded content a name so it can be referenced or targeted from a link or script. Rarely needed for modern PDFs/images.

You may also see a <param> tag inside older <object> examples online. It's a legacy child element used to pass settings to plugins (like old Java applets), written as <param name="..." value="...">. Modern PDFs and images don't need it, but recognising it helps when reading older code.

8. <iframe> — The Iframe Tag

Definition

The <iframe> tag embeds an entire separate webpage inside the current page — like a window inside a window. It is the tag behind embedded YouTube videos, Google Maps, and online forms.

Syntax

```
<iframe src="https://example.com"
  width="560" height="315" title="description">
</iframe>
```

Attributes

Attribute	Type / Values	What It Does
src	URL	The webpage being embedded. Required.
width / height	Number (pixels)	Sets the display size of the embedded frame.
title	Text	A short description of the frame's content. Important for accessibility. Should always be included.
allow	Permissions list	Grants specific permissions to the embedded page, e.g. "fullscreen; autoplay".
allowfullscreen	Boolean	Lets the embedded content be shown in fullscreen mode.
loading	"lazy" or "eager"	"lazy" delays loading the frame until the user scrolls near it.
sandbox	Restriction list	Restricts what the embedded page is allowed to do, for extra security.

Example

```
<iframe src="https://www.youtube.com/embed/VIDEO_ID"
  width="560" height="315"
  title="YouTube video player" allowfullscreen>
</iframe>
```

Tip: <iframe> always needs a closing tag, even though it's often empty: <iframe ...></iframe>, not <iframe ... />.

NEW — More <iframe> Attributes: name, referrerpolicy, srcdoc

Attribute	Type / Values	What It Does
name	Text	Lets a link's target or a form's target point content into this specific iframe.
referrerpolicy	e.g. "no-referrer"	Controls how much referrer information is sent when loading the embedded page.
srcdoc	Raw HTML	Displays inline HTML you write directly, instead of loading a separate URL. Overrides src when both are present.

NEW — Example: srcdoc

```
<iframe title="quick demo" width="400" height="150"
  srcdoc="<h2>Hello!</h2><p>This HTML is inline.</p>">
</iframe>
```

Tip: srcdoc is handy for classroom demos — you can show a working iframe with no external file or live URL needed.

NEW — MIME Types, in Plain Terms

A MIME type is a short label that tells the browser what kind of file it's dealing with, written as category/format. You've already used MIME types in the type attribute of <source>, <embed>, and <object> without them being named as one shared idea — it's the same concept every time.

File Type	MIME Type
MP3 audio	audio/mpeg
OGG audio	audio/ogg
MP4 video	video/mp4
WebM video	video/webm
PDF document	application/pdf
Subtitle file (.vtt)	text/vtt
JPEG image	image/jpeg
WebP image	image/webp

NEW — Choosing Between <embed>, <object>, and <iframe>

All three "embed something" into the page — the deciding factor is what you're embedding and whether you need a fallback message.

If you need to...	Use this tag
Embed another live webpage (YouTube, Google Maps, a form)	<iframe>
Embed a file (like a PDF) and show a fallback message if it can't display	<object>
Embed a file (like a PDF) with no fallback message needed	<embed>

Quick Comparison — All Media Tags at a Glance

Tag	What It's For	Closing Tag?	Common Real-World Use
	Displays an image	No	Photos, icons, illustrations
<picture>	Shows different images for different screens	Yes	Responsive images, WebP with JPG fallback
<audio>	Plays sound	Yes	Podcasts, music players, voice notes
<video>	Plays video	Yes	Lessons, movie clips, product demos
<source>	Offers alternate media formats	No	Cross-browser audio/video support
<track>	Adds subtitles/captions	No	Accessibility, multi-language captions
<embed>	Embeds an external file	No	PDFs, simple external widgets
<object>	Embeds external content with a fallback	Yes	PDFs, documents, with backup message
<iframe>	Embeds another webpage	Yes	YouTube videos, Google Maps, forms

Tip: The <picture> row is highlighted because it's new this revision — same idea as the rest of the table, just for choosing between whole images rather than sizes of the same image.

Common Mistakes to Avoid

- Forgetting alt text on tags — this breaks accessibility and hurts SEO.
- Forgetting the closing tag on <audio>, <video>, <object>, and <iframe>.
- Leaving out type on a <source> tag, which can stop browsers from picking the correct file.
- Expecting autoplay with sound to always work — most browsers block it unless muted is also set.
- Leaving out title on an <iframe> — this is a common accessibility mistake.
- Providing only one file format for audio/video — not every browser supports every format.
- Forgetting playsinline on <video> — on iPhone Safari, playback jumps to fullscreen the moment it starts.
- Reaching for <embed> when you actually need a fallback message — <embed> can't hold fallback content, so use <object> instead.

Practical 1: Build a Multimedia Profile Page

Let's combine everything from this guide into one real webpage. We'll build a simple "My Profile" page that uses every tag covered above: image, audio, video, source, track, embed, object, and iframe.

What You'll Need

Save all of these files in the same folder as your HTML file:

- A profile picture — profile.jpg
- A short audio clip — intro.mp3 and intro.ogg
- A short video clip — about.mp4 and about.webm, plus a poster image poster.jpg
- A caption file for the video — captions.vtt
- A PDF file — resume.pdf

The Code

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
<meta charset="UTF-8">
<title>My Multimedia Profile</title>
</head>
<body>
<h1>My Multimedia Profile</h1>

<!-- 1. IMAGE -->


<!-- 2. AUDIO -->
<h2>About Me (Audio Intro)</h2>
<audio controls>
  <source src="intro.mp3" type="audio/mpeg">
  <source src="intro.ogg" type="audio/ogg">
  Your browser does not support the audio element.
</audio>

<!-- 3. VIDEO with SOURCE + TRACK -->
<h2>Video Introduction</h2>
<video width="480" height="270" controls poster="poster.jpg">
  <source src="about.mp4" type="video/mp4">
  <source src="about.webm" type="video/webm">
  <track src="captions.vtt" kind="captions"
    srclang="en" label="English" default>
  Your browser does not support the video tag.
</video>

<!-- 4. EMBED -->
<h2>My Resume</h2>
<embed src="resume.pdf" type="application/pdf"
  width="500" height="400">

<!-- 5. OBJECT with fallback -->
<h2>My Resume (with fallback)</h2>
<object data="resume.pdf" type="application/pdf"
  width="500" height="400">
  <p>Can't display the PDF.
  <a href="resume.pdf">Download it here</a>.</p>
</object>

<!-- 6. IFRAME -->
<h2>Find Me on the Map</h2>
<iframe
src="https://maps.google.com/maps?q=New%20Delhi&output=embed"
width="500" height="350" title="Map location"
loading="lazy"></iframe>

<footer>
<p>&copy; 2026 My Multimedia Profile</p>
</footer>

</body>
</html>

```

Try It Yourself

1. Save the code above as profile.html in a new folder.
2. Add or download the media files listed above into that same folder.

3. Open profile.html in your browser and check that the image, audio, video, PDF, and map all appear.
4. Add the loop attribute to the <audio> tag and reload the page — what changes?
5. Change the <video> tag's controls to autoplay muted loop and reload the page.
6. Create your own captions.vtt file and add a second <track> tag in a different language.
7. Remove the alt attribute from the tag, then imagine you are using a screen reader — what would happen?

NEW — Practical 2: Debugging Lab

Each snippet below has exactly one bug. Find it and write down the fix before checking the answer key at the end — this is the skill you'll actually use once you're writing your own pages.

NEW — Snippet 1 — what's wrong here?

```
<audio controls>
  <source src="theme-song.mp3" type="video/mp4">
  <source src="theme-song.ogg" type="video/mp4">
</audio>
```

NEW — Snippet 2 — what's wrong here?

```

```

NEW — Snippet 3 — what's wrong here?

```
<video src="promo.mp4" autoplay controls></video>
```

NEW — Snippet 4 — what's wrong here?

```
<iframe src="https://example.com/map" width="400" height="300" />
```

NEW — Snippet 5 — what's wrong here?

```
<embed src="policy.pdf" type="application/pdf" width="500" height="400">
  <p>Sorry, your browser can't display this PDF.
  <a href="policy.pdf">Download it here</a>.</p>
</embed>
```

NEW — Snippet 6 — what's wrong here?

```
<iframe src="https://www.youtube.com/embed/xyz" width="560" height="315"></iframe>
```

NEW — Answer Key

Snippet	The Bug and the Fix
1	The type values say video/mp4 but the files are audio. Should be type="audio/mpeg" and type="audio/ogg".
2	Missing the alt attribute — required for accessibility and for when the image fails to load.
3	autoplay is used without muted, so most browsers will silently block playback. Add muted.
4	<iframe> was written as self-closing (/>). It must have a full closing tag: <iframe ...></iframe>.
5	<embed> can never show fallback content — it's an empty tag, so the <p> inside it is ignored. Use <object> instead.
6	Missing the title attribute — a common accessibility miss on <iframe>.

NEW — Practical 3: Level-Up Build

Go back to your Practical 1 profile page and upgrade three parts of it, using only what's in this revision:

- Replace the plain `<img>` profile picture with a `<picture>` tag offering a smaller image for narrow screens.
- Add `playsinline` to your `<video>` tag so it behaves properly when opened on a phone.
- Add a new small section that uses an `<iframe>` with `srcdoc` instead of a `src` URL — no external file needed.

NEW — Starter hint for the `srcdoc` section

```
<h2>A Quick Note</h2>
<iframe title="quick note" width="400" height="120"
  srcdoc="<p>Thanks for visiting my profile page!</p>">
</iframe>
```

Tip: Because `srcdoc` needs no external file, this is the fastest section to test — start here if you want a quick win before tackling `<picture>`.

Tip: For more detail on any tag, search its name on MDN Web Docs (developer.mozilla.org) — it's the most reliable free HTML reference online.
